Работа с данными в ASP.NET

Введение

В данной лекции рассматривается организация работы с данными в ASP.NET Web API. Будут изучены подходы к проектированию архитектуры приложения, подключение базы данных SQLite, использование паттерна DTO совместно с AutoMapper, валидация входящих данных, а также работа с файлами — их загрузка на сервер и возврат клиенту различными способами.

1. Подключение БД

Прежде чем приступить к написанию кода, необходимо определить архитектуру проекта. В рамках данной лекции используется следующая схема взаимодействия слоёв: контроллер принимает HTTP-запрос и обращается к репозиторию; репозиторий работает напрямую с `DbContext`, который взаимодействует с базой данных SQLite. Структура папок проекта организована следующим образом:

```
/Models
/DTOs
/Repositories
/Controllers
```

Устанавливаем необходимые пакеты для работы с SQLite:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

В качестве примера на протяжении всей лекции используется сущность `Product`. Модель описывается следующим образом:

```
// Models/Product.cs
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string? ImagePath { get; set; }
}
```

Далее создаётся контекст базы данных:

```
// Data/AppDbContext.cs
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Product> Products => Set<Product>();
}
```

Строка подключения указывается в файле `appsettings.json`:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  }
}
```

Интерфейс и реализация репозитория выглядят следующим образом:

```
// Repositories/IProductRepository.cs
public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
    Task AddAsync(Product product);
    Task UpdateAsync(Product product);
    Task DeleteAsync(Product product);
}
```

```
// Repositories/ProductRepository.cs
public class ProductRepository : IProductRepository
{
    private readonly AppDbContext _context;

    public ProductRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<IEnumerable<Product>> GetAllAsync()
    {
        return await _context.Products.ToListAsync();
    }

    public async Task<Product?> GetByIdAsync(int id)
    {
```

```
        return await _context.Products.FindAsync(id);
    }

    public async Task AddAsync(Product product)
    {
        await _context.Products.AddAsync(product);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Product product)
    {
        _context.Products.Update(product);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(Product product)
    {
        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
    }
}
```

Регистрация зависимостей

После настройки контекста зависимости регистрируются в `Program.cs`. `DbContext` и репозиторий регистрируются с временем жизни `Scoped`, то есть один экземпляр создаётся на один HTTP-запрос. Это необходимо, поскольку `DbContext` не является потокобезопасным и не должен использоваться совместно между запросами.

```
// Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("DefaultConnection")
        )
    );

builder.Services.AddScoped<IProductRepository, ProductRepository>();
```

После регистрации контекста создаем и применяем миграцию для формирования схемы базы данных:

```
dotnet ef migrations add Init
dotnet ef database update
```

2. DTO и AutoMapper

При разработке Web API возникает вопрос: что именно возвращать клиенту в ответ на запрос и что принимать от него? Наивный подход — использовать доменные модели напрямую — влечёт за собой ряд серьёзных проблем.

1. Утечка чувствительных данных. Если модель пользователя содержит поле `PasswordHash`, оно будет сериализовано и отправлено клиенту вместе с остальными данными, если явно это не запретить.
2. Циклические ссылки при сериализации. Entity Framework формирует навигационные свойства между связанными сущностями, что при сериализации приводит к бесконечной рекурсии. Например, `Product` ссылается на `Category`, а `Category` содержит коллекцию `Products`.
3. Жёсткая связь между контрактом API и структурой базы данных. Если изменится схема БД, это напрямую затронет ответы API и может сломать клиентские приложения.

Решением всех перечисленных проблем является паттерн **DTO (Data Transfer Object)** — отдельные классы, которые описывают исключительно то, что нужно передать через API.

Request и Response DTO

Принято разделять DTO на два типа: входящий (`RequestDto`) и исходящий (`ResponseDto`). Входящий DTO описывает данные, которые клиент отправляет на сервер. Исходящий DTO описывает данные, которые сервер возвращает клиенту. Разделение необходимо, поскольку эти наборы полей, как правило, различаются: например, поле `Id` не должно присутствовать во входящей модели при создании сущности, так как оно генерируется на стороне сервера.

```
// DTOs/ProductRequestDto.cs
public class ProductRequestDto
{
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
}
```

```
// DTOs/ProductResponseDto.cs
public class ProductResponseDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
```

```
    public decimal Price { get; set; }
}
```

Валидация через Data Annotations

Валидация входящих данных выполняется посредством атрибутов из пространства имён `System.ComponentModel.DataAnnotations`, которые размещаются на свойствах `RequestDto`. Наиболее часто используемые атрибуты представлены ниже:

```
// DTOs/ProductRequestDto.cs
public class ProductRequestDto
{
    [Required(ErrorMessage = "Название продукта обязательно для
заполнения")]
    [MaxLength(100, ErrorMessage = "Название не должно превышать 100
символов")]
    public string Name { get; set; } = string.Empty;

    [MaxLength(500, ErrorMessage = "Описание не должно превышать 500
символов")]
    public string Description { get; set; } = string.Empty;

    [Range(0.01, 999999.99, ErrorMessage = "Цена должна быть в диапазоне от
0.01 до 999999.99")]
    public decimal Price { get; set; }
}
```

Атрибут `[ApiController]`, который устанавливается на контроллере, обеспечивает автоматическую проверку модели. Если входящие данные не прошли валидацию, ASP.NET автоматически возвращает ответ `400 Bad Request` с подробным описанием ошибок, не передавая управление в метод контроллера. Ответ имеет следующую структуру:

```
{
  "errors": {
    "Name": ["Название продукта обязательно для заполнения"],
    "Price": ["Цена должна быть в диапазоне от 0.01 до 999999.99"]
  },
  "status": 400,
  "title": "One or more validation errors occurred."
}
```

Ручной маппинг

После того как DTO определены, возникает задача преобразования данных между ними и доменными моделями. Этот процесс называется маппингом. Маппинг необходим в двух направлениях: при получении запроса от клиента входящий DTO преобразуется в сущность для сохранения в базе данных, а при формировании ответа сущность преобразуется в исходящий DTO.

Наиболее очевидный способ реализации маппинга — ручное присвоение полей:

```
// Преобразование RequestDto в сущность
var product = new Product
{
    Name = dto.Name,
    Description = dto.Description,
    Price = dto.Price
};

// Преобразование сущности в ResponseDto
var responseDto = new ProductResponseDto
{
    Id = product.Id,
    Name = product.Name,
    Description = product.Description,
    Price = product.Price
};
```

При небольшом количестве полей такой подход вполне приемлем и прозрачен. Однако в реальных проектах сущности нередко содержат десятки свойств, а количество самих сущностей исчисляется десятками. В таком случае ручной маппинг становится громоздким, однообразным и трудно поддерживаемым: при добавлении нового поля в модель необходимо не забыть обновить маппинг во всех местах, где он используется. Для решения этой проблемы применяются специализированные библиотеки, которые автоматизируют процесс преобразования объектов. Одна из наиболее распространённых — AutoMapper, которая будет рассмотрена в следующем разделе.

AutoMapper

AutoMapper — библиотека, которая автоматизирует преобразование объектов одного типа в другой. Для её подключения необходимо установить пакет:

```
dotnet add package AutoMapper
```

Основным понятием в AutoMapper является профиль — класс, в котором описываются правила маппинга между типами. Профиль наследуется от базового класса `Profile`:

```
// Mappings/ProductMappingProfile.cs
public class ProductMappingProfile : Profile
{
    public ProductMappingProfile()
    {
        // Маппинг из RequestDto в сущность
        CreateMap<ProductRequestDto, Product>();

        // Маппинг из сущности в ResponseDto
        CreateMap<Product, ProductResponseDto>();
    }
}
```

Аналогично делаются классы для остальных DTO. В случае если имена свойств в источнике и целевом типе совпадают, AutoMapper выполняет маппинг автоматически. Если необходимо задать особое правило для конкретного поля, используется метод `ForMember`:

```
CreateMap<Product, ProductResponseDto>()
    .ForMember(dest => dest.Name,
               opt => opt.MapFrom(src => src.Name.Trim()));
```

Далее необходимо зарегистрировать AutoMapper в DI-контейнере. Метод `AddAutoMapper` принимает сборку, в которой следует искать классы, которые наследуются от `Profile`:

```
// Program.cs
builder.Services.AddAutoMapper(typeof(Program));
```

После регистрации интерфейс `IMapper` становится доступен для внедрения в любой класс через конструктор, после чего его можно использовать для преобразования DTO в доменную модель и обратно, с помощью метода `Map`.

3. Контроллер

В контроллер через конструктор передаются репозиторий и маппер:

```
// Controllers/ProductsController.cs
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductRepository _repository;
    private readonly IMapper _mapper;

    public ProductsController(IProductRepository repository, IMapper mapper)
    {
        _repository = repository;
        _mapper = mapper;
    }
}
```

Получение данных

Метод получения всех записей обращается к репозиторию и преобразует коллекцию сущностей в коллекцию исходящих DTO, через метод `Map`:

```
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var products = await _repository.GetAllAsync();
    var response = _mapper.Map<IEnumerable<ProductResponseDto>>(products);
    return Ok(response);
}
```

Метод получения записи по идентификатору дополнительно проверяет, была ли найдена запись, и возвращает `404 Not Found` в случае её отсутствия:

```
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var product = await _repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    var response = _mapper.Map<ProductResponseDto>(product);
    return Ok(response);
}
```

Создание записи

При создании новой записи входящий DTO преобразуется в сущность, сохраняется через репозиторий, после чего сущность преобразуется в исходящий DTO и возвращается клиенту. Для ответа используется статус `201 Created`, который согласно стандарту HTTP должен сопровождаться заголовком `Location`, указывающим на созданный ресурс:

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] ProductRequestDto dto)
{
    var product = _mapper.Map<Product>(dto);
    await _repository.AddAsync(product);

    var response = _mapper.Map<ProductResponseDto>(product);
    return CreatedAtAction(nameof(GetById), new { id = product.Id },
response);
}
```

Обновление записи

При обновлении записи используется перегрузка `_mapper.Map(src, dest)`, которая применяет значения из DTO к уже существующему объекту сущности, не создавая новый экземпляр. Это позволяет сохранить поля, которые не присутствуют во входящем DTO, например `ImagePath`:

```
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, [FromBody] ProductRequestDto
dto)
{
    var product = await _repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    _mapper.Map(dto, product);
    await _repository.UpdateAsync(product);
    return NoContent();
}
```

Успешное обновление возвращает статус `204 No Content` — запрос был выполнен, но тело ответа отсутствует, поскольку клиент уже располагает актуальными данными, которые он сам и передал.

Удаление записи

```
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var product = await _repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    await _repository.DeleteAsync(product);
    return NoContent();
}
```

4. Хранение файлов

При разработке Web API нередко возникает необходимость работы с файлами — изображениями, документами, архивами и прочими бинарными данными. Прежде чем приступать к реализации, необходимо принять архитектурное решение: где именно эти файлы будут храниться. Существует два основных подхода.

Хранение файлов в базе данных

Файл может быть сохранён непосредственно в базе данных в виде бинарных данных. В терминах реляционных баз данных такой тип данных называется BLOB (Binary Large Object). В контексте Entity Framework и C# это соответствует типу `byte[]`:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public byte[]? Image { get; set; }
}
```

Данный подход обладает рядом преимуществ. Целостность данных обеспечивается автоматически — файл и связанная с ним запись существуют в рамках одной транзакции, поэтому не может возникнуть ситуации, когда запись в БД есть, а файл отсутствует. Резервное копирование упрощается, поскольку достаточно создать резервную копию одной базы данных.

Вместе с тем у этого подхода есть существенные недостатки. База данных быстро увеличивается в размере, что негативно сказывается на производительности запросов, не связанных с файлами. Нагрузка на СУБД возрастает, поскольку она вынуждена обрабатывать большие объёмы бинарных данных. Отдача файла клиенту усложняется — необходимо сначала загрузить данные из БД в память, а затем передать их в ответ.

Хранение файлов в файловой системе

Альтернативный подход — сохранять файлы на диске, а в базе данных хранить только путь к файлу или его имя. Именно этот подход будет использоваться в рамках данной лекции:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? ImagePath { get; set; }
}
```

Преимущества данного подхода очевидны. Производительность отдачи файлов значительно выше — файл можно передавать напрямую с диска, минуя базу данных. Размер БД остаётся небольшим, поскольку она хранит только метаданные. Файлы можно обслуживать через встроенный механизм статических файлов ASP.NET, не нагружая контроллеры.

Основным недостатком является необходимость самостоятельно следить за синхронизацией: если запись удаляется из БД, соответствующий файл на диске необходимо удалять отдельно. В противном случае на диске накапливаются неиспользуемые файлы.

Практическая рекомендация

Выбор подхода определяется требованиями конкретного проекта. В качестве ориентира можно использовать следующие рекомендации:

- Если файлы небольшие, критичны к целостности и их количество невелико — хранение в БД допустимо.
- Если речь идёт об изображениях, медиафайлах или файлах, загружаемых пользователями, предпочтительно хранить их в файловой системе.
- В production-окружении для хранения файлов, как правило, используются объектные хранилища, такие как Amazon S3 или MinIO. Они сочетают в себе преимущества файловой системы и предоставляют дополнительные возможности: репликацию, разграничение доступа и т.д.

5. Загрузка файлов на сервер

Для загрузки файлов используется тип `IFormFile`. В отличие от обычных JSON-запросов, файлы передаются в теле запроса в формате `multipart/form-data`. По этой причине параметр помечается атрибутом `[FromForm]`, а не `[FromBody]`.

Для загрузки файла создаётся отдельный контроллер `FilesController`:

```
// Controllers/FilesController.cs
[ApiController]
[Route("api/[controller]")]
public class FilesController : ControllerBase
{
    private readonly IWebHostEnvironment _environment;

    public FilesController(IWebHostEnvironment environment)
    {
        _environment = environment;
    }

    [HttpPost("upload")]
    public async Task<IActionResult> Upload([FromForm] IFormFile file)
    {
        // реализация рассматривается далее
    }
}
```

Если необходимо принять несколько файлов одновременно, параметр объявляется как `IEnumerable<IFormFile>`:

```
[HttpPost("upload-multiple")]
public async Task<IActionResult> UploadMultiple([FromForm]
IEnumerable<IFormFile> files)
{
    // обработка нескольких файлов
}
```

Интерфейс `IWebHostEnvironment` предоставляет доступ к путям файловой системы приложения, в частности к свойству `WebRootPath` — абсолютному пути к папке `wwwroot`.

Папка `wwwroot` является корневой директорией для статических файлов в ASP.NET-приложении. Именно в ней принято размещать файлы, которые должны быть доступны клиентам: изображения, стили, скрипты и прочие ресурсы. Файлы, размещённые за пределами `wwwroot`, по умолчанию недоступны по HTTP.

Валидация загружаемого файла

Прежде чем сохранять файл на диск, необходимо убедиться в его допустимости. Валидация должна включать проверку размера файла, его расширения и MIME-типа.

Проверка расширения выполняется через `Path.GetExtension`, MIME-тип доступен через свойство `ContentType` объекта `IFormFile`. Важно выполнять обе проверки, поскольку расширение файла может быть изменено вручную, тогда как MIME-тип определяется содержимым:

```
private static readonly string[] AllowedExtensions = { ".jpg", ".jpeg",
".png", ".webp" };
private static readonly string[] AllowedMimeTypes = { "image/jpeg",
"image/png", "image/webp" };
private const long MaxFileSize = 5 * 1024 * 1024; // 5 МБ

private IActionResult? ValidateFile(IFormFile file)
{
    if (file.Length == 0)
        return BadRequest("Файл пустой.");

    if (file.Length > MaxFileSize)
        return BadRequest("Размер файла превышает допустимый предел в 5
МБ.");

    var extension = Path.GetExtension(file.FileName).ToLowerInvariant();
    if (!AllowedExtensions.Contains(extension))
        return BadRequest("Недопустимое расширение файла.");

    if (!AllowedMimeTypes.Contains(file.ContentType.ToLowerInvariant()))
        return BadRequest("Недопустимый тип файла.");

    return null;
}
```

Метод возвращает `null`, если файл прошёл все проверки, либо результат с ошибкой, который затем возвращается из метода контроллера.

Помимо валидации на уровне кода, можно ограничить максимальный размер тела запроса на уровне конкретного action с помощью атрибута `[RequestSizeLimit]`:

```
[HttpPost("upload")]
[RequestSizeLimit(5 * 1024 * 1024)]
public async Task<IActionResult> Upload([FromForm] IFormFile file)
{
    // ...
}
```

Сохранение файла на диск

После успешной валидации файл сохраняется в папку `wwwroot/uploads`. Оригинальное имя файла использовать не рекомендуется по нескольким причинам: оно может содержать кириллицу, пробелы или специальные символы, которые некорректно обрабатываются в URL, а кроме того, совпадение имён приведёт к перезаписи уже существующих файлов. Поэтому для каждого загружаемого файла генерируется уникальное имя на основе `Guid`:

```
[HttpPost("upload")]
[RequestSizeLimit(5 * 1024 * 1024)]
public async Task<IActionResult> Upload([FromForm] IFormFile file)
{
    var validationError = ValidateFile(file);
    if (validationError is not null)
        return validationError;

    // Формирование уникального имени файла
    var extension = Path.GetExtension(file.FileName).ToLowerInvariant();
    var fileName = $"{Guid.NewGuid()}{extension}";

    // Формирование пути для сохранения
    var uploadsFolder = Path.Combine(_environment.WebRootPath, "uploads");

    // Создание папки, если она не существует
    if (!Directory.Exists(uploadsFolder))
        Directory.CreateDirectory(uploadsFolder);

    var filePath = Path.Combine(uploadsFolder, fileName);

    // Сохранение файла на диск
    using var stream = new FileStream(filePath, FileMode.Create);
    await file.CopyToAsync(stream);

    // Возврат относительного пути, который сохраняется в БД
    var relativePath = Path.Combine("uploads", fileName);
    return Ok(new { path = relativePath });
}
```

Следует обратить внимание на использование `Path.Combine` вместо ручной конкатенации строк. Этот метод корректно обрабатывает разделители пути на всех операционных системах, что важно при развёртывании приложения на Linux-серверах.

Привязка файла к сущности

В реальном приложении относительный путь к файлу сохраняется в базе данных как поле сущности. Применительно к модели `Product` из предыдущих разделов загрузка изображения для конкретного продукта выглядит следующим образом:

```
[HttpPost("{id}/image")]
[RequestSizeLimit(5 * 1024 * 1024)]
public async Task<IActionResult> UploadProductImage(
    int id,
    [FromForm] IFormFile file,
    [FromServices] IProductRepository repository)
{
    var product = await repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    var validationError = ValidateFile(file);
    if (validationError is not null)
        return validationError;

    var extension = Path.GetExtension(file.FileName).ToLowerInvariant();
    var fileName = $"{Guid.NewGuid()}{extension}";
    var uploadsFolder = Path.Combine(_environment.WebRootPath, "uploads");

    if (!Directory.Exists(uploadsFolder))
        Directory.CreateDirectory(uploadsFolder);

    var filePath = Path.Combine(uploadsFolder, fileName);

    using var stream = new FileStream(filePath, FileMode.Create);
    await file.CopyToAsync(stream);

    product.ImagePath = Path.Combine("uploads", fileName);
    await repository.UpdateAsync(product);

    return Ok(new { path = product.ImagePath });
}
```

Атрибут `[FromServices]` указывает ASP.NET, что данный параметр метода должен быть получен из DI-контейнера, а не из тела запроса или маршрута. Обычно зависимости инжектируются через конструктор контроллера, однако если зависимость требуется только в одном конкретном методе, использование `[FromServices]` позволяет не добавлять её в конструктор и тем самым не увеличивать связность класса. В данном примере `IProductRepository` нужен исключительно для привязки файла к продукту, поэтому его удобнее получить непосредственно в параметрах метода.

6. Возврат файлов клиенту

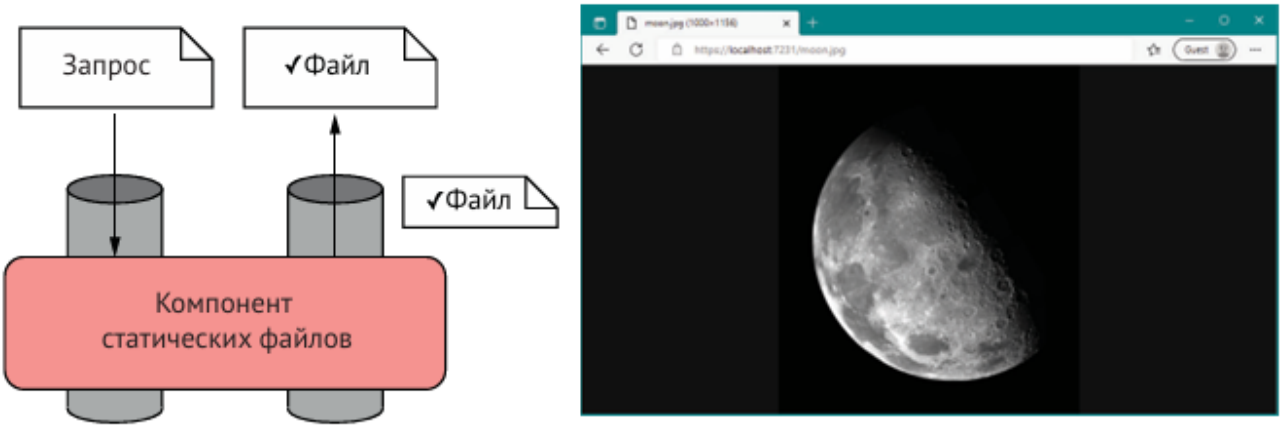
После того как файл загружен на сервер и путь к нему сохранён в базе данных, необходимо обеспечить возможность получения этого файла клиентом. Существует три основных подхода: отдача через механизм статических файлов, возврат через `PhysicalFile` и стриминг. Каждый из них имеет свои преимущества и подходит для различных сценариев.

Статическая отдача через UseStaticFiles

Первый и наиболее простой подход — использование встроенного middleware `UseStaticFiles`. При его подключении ASP.NET автоматически обрабатывает запросы к файлам, расположенным в папке `wwwroot`, без участия контроллеров. Для подключения достаточно добавить одну строку в `Program.cs`:

```
// Program.cs
app.UseStaticFiles();
```

После этого любой файл из папки `wwwroot/uploads` становится доступен по URL вида `https://example.com/uploads/filename.jpg`. Контроллер при этом не задействуется — запрос обрабатывается на уровне middleware.



1. Компонент статических файлов обрабатывает запрос, возвращая запрошенный файл

2. Файловый поток отправляется обратно через конвейер промежуточного ПО и в браузер

3. Браузер отображает файл, возвращенный в ответе

При таком подходе контроллер возвращает клиенту не сам файл, а URL для его получения. Формирование URL выполняется следующим образом:

```
[HttpGet("{id}/image-url")]
public async Task<IActionResult> GetImageUrl(
    int id,
    [FromServices] IProductRepository repository)
{
    var product = await repository.GetByIdAsync(id);
```

```
    if (product is null)
        return NotFound();

    if (product.ImagePath is null)
        return NotFound("У данного продукта отсутствует изображение.");

    var url = $"{Request.Scheme}://{Request.Host}/{product.ImagePath}";
    return Ok(new { url });
}
```

Преимущество данного подхода — высокая производительность и простота реализации. Существенный недостаток — полное отсутствие контроля доступа: любой пользователь, знающий URL файла, сможет его получить. Для публичных ресурсов, таких как изображения товаров, это приемлемо, однако для файлов с ограниченным доступом такой подход неприменим.

Возврат файла через `PhysicalFile`

Второй подход предполагает отдачу файла непосредственно через метод контроллера. Это позволяет выполнить проверку прав доступа, логировать факт скачивания или выполнить любую другую логику перед отдачей файла.

Метод `PhysicalFile` принимает абсолютный путь к файлу на диске и MIME-тип, после чего ASP.NET самостоятельно читает файл и передаёт его в тело ответа:

```
[HttpGet("{id}/image")]
public async Task<IActionResult> GetImage(
    int id,
    [FromServices] IProductRepository repository)
{
    var product = await repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    if (product.ImagePath is null)
        return NotFound("У данного продукта отсутствует изображение.");

    var absolutePath = Path.Combine(_environment.WebRootPath,
product.ImagePath);
    if (!System.IO.File.Exists(absolutePath))
        return NotFound("Файл не найден на сервере.");

    var contentType = "image/jpeg";
    return PhysicalFile(absolutePath, contentType);
}
```

Если файл хранится не на диске, а в базе данных в виде массива байт, используется перегрузка метода `File`:

```
var bytes = product.ImageData; // byte[] из БД
return File(bytes, "image/jpeg");
```

По умолчанию браузер попытается отобразить файл, если поддерживает соответствующий MIME-тип. Если требуется принудительно инициировать скачивание файла, необходимо передать имя файла третьим параметром — это добавит заголовок `Content-Disposition: attachment` в ответ:

```
return PhysicalFile(absolutePath, "image/jpeg", "product-image.jpg");
```

Недостаток данного подхода состоит в том, что файл полностью загружается в память перед отправкой клиенту. При работе с большими файлами это может привести к значительному потреблению оперативной памяти сервера.

Стриминг файла

Третий подход — передача файла потоком с помощью `FileStreamResult`. В отличие от предыдущего подхода, файл не загружается в память целиком, а читается с диска и передаётся клиенту последовательно, небольшими фрагментами. Это делает стриминг предпочтительным выбором для больших файлов — видео, архивов и прочих ресурсов значительного объёма:

```
[HttpGet("{id}/image-stream")]
public async Task<IActionResult> GetImageStream(
    int id,
    [FromServices] IProductRepository repository)
{
    var product = await repository.GetByIdAsync(id);
    if (product is null)
        return NotFound();

    if (product.ImagePath is null)
        return NotFound("У данного продукта отсутствует изображение.");

    var absolutePath = Path.Combine(_environment.WebRootPath,
product.ImagePath);
    if (!System.IO.File.Exists(absolutePath))
        return NotFound("Файл не найден на сервере.");

    var stream = new FileStream(absolutePath, FileMode.Open,
FileAccess.Read);
    return new FileStreamResult(stream, "image/jpeg");
}
```

Следует обратить внимание на то, что в данном случае `FileStream` не оборачивается в `using`. Это сделано намеренно: ASP.NET самостоятельно закрывает поток после завершения передачи данных клиенту. Если закрыть поток досрочно через `using`, передача файла будет прервана.

При работе с видеофайлами браузер отправляет так называемые `Range`-запросы — он запрашивает не весь файл целиком, а отдельные его фрагменты, что позволяет перематывать видео без полной загрузки. Для поддержки таких запросов необходимо установить свойство `EnableRangeProcessing`:

```
return new FileStreamResult(stream, "video/mp4")
{
    EnableRangeProcessing = true
};
```

Если мы хотим чтобы файл автоматически скачивался нужно установить свойство `FileDownloadName`:

```
return new FileStreamResult(stream, "video/mp4")
{
    EnableRangeProcessing = true,
    FileDownloadName = "video.mp4"
};
```

Сравнение подходов

Выбор подхода определяется требованиями конкретного сценария. В таблице ниже представлено сравнение трёх рассмотренных методов:

	Статика	**PhysicalFile**	**Стриминг**
Производительность	Высокая	Средняя	Высокая
Контроль доступа	Отсутствует	Есть	Есть
Потребление памяти	Минимальное	Высокое	Минимальное
Сложность реализации	Минимальная	Низкая	Низкая
Подходит для больших файлов	Да	Нет	Да
Типичный сценарий	Публичные изображения	Файлы с проверкой доступа, файлы из БД	Видео, большие архивы